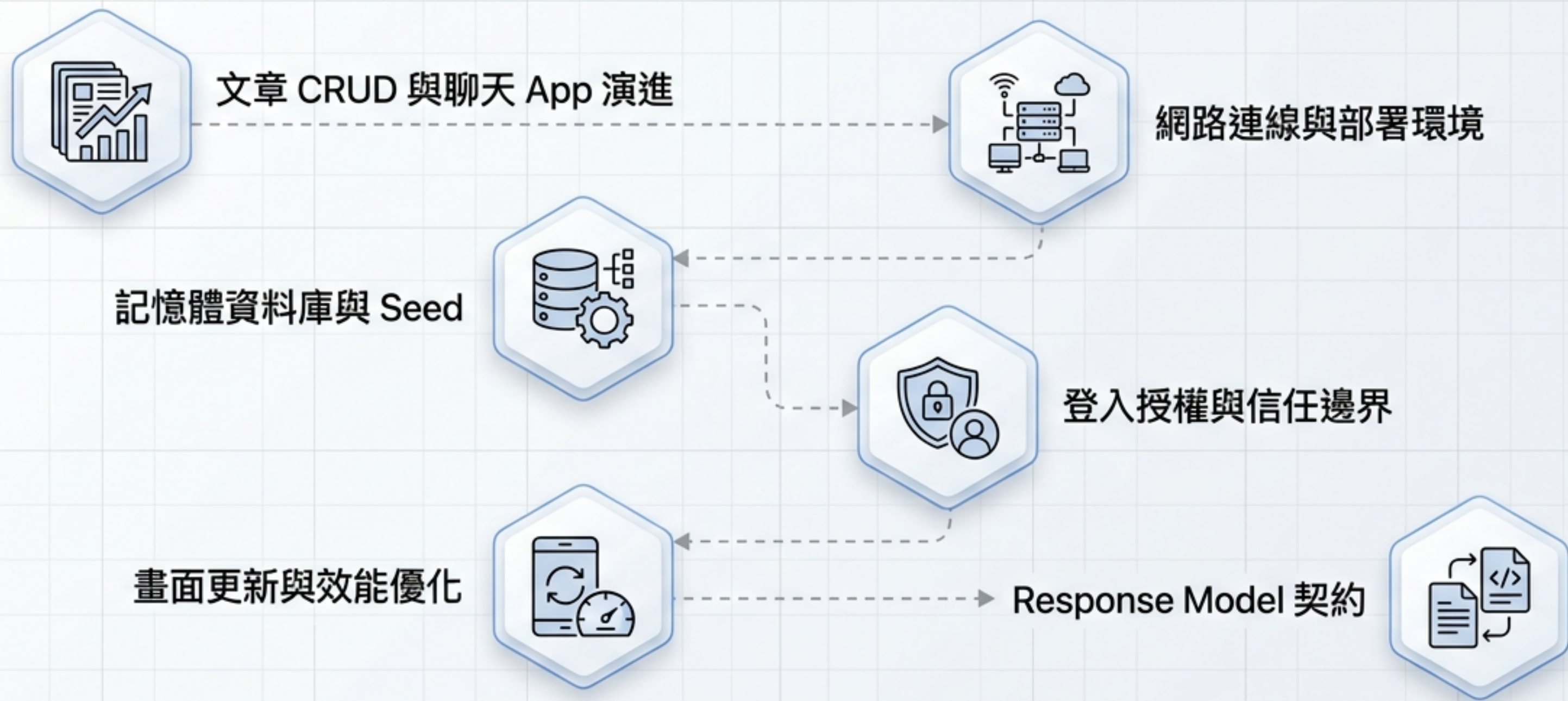


現代全端開發架構：從 CRUD 到聊天 App 的實務銜接

React Native 串接 FastAPI 系統指引 —— 環境部署、資料狀態、授權機制與前端優化



系統複雜度的躍升：單一模型 vs 多模型應用

Phase 1: 基礎打底

- 核心目標：fullstack 01-44
- 資料模型：單一模型
Post (文章)
- 實作流程：
列表 → 新增 → 詳細 → 編輯 → 刪除
- 技術焦點：
基礎 GET / POST / PUT / DELETE 練習

系統複雜度提升

Phase 2: 實務應用

- 核心目標：fullstack 45-84
- 資料模型：多重關聯模型
User, Friendship, Message
- 實作流程：
登入狀態 → 好友關係 → 聊天列表 → 即時收發
- 技術焦點：
跨模型關聯、狀態管理、實務應用場景

API 任務重構：對應真實商業邏輯

文章 CRUD 邏輯

[GET/POST/PUT/DELETE] /posts

單純的增刪改查，缺乏業務邏輯深度

聊天 App 業務邏輯

[POST] /auth/register & /auth/login 身管理

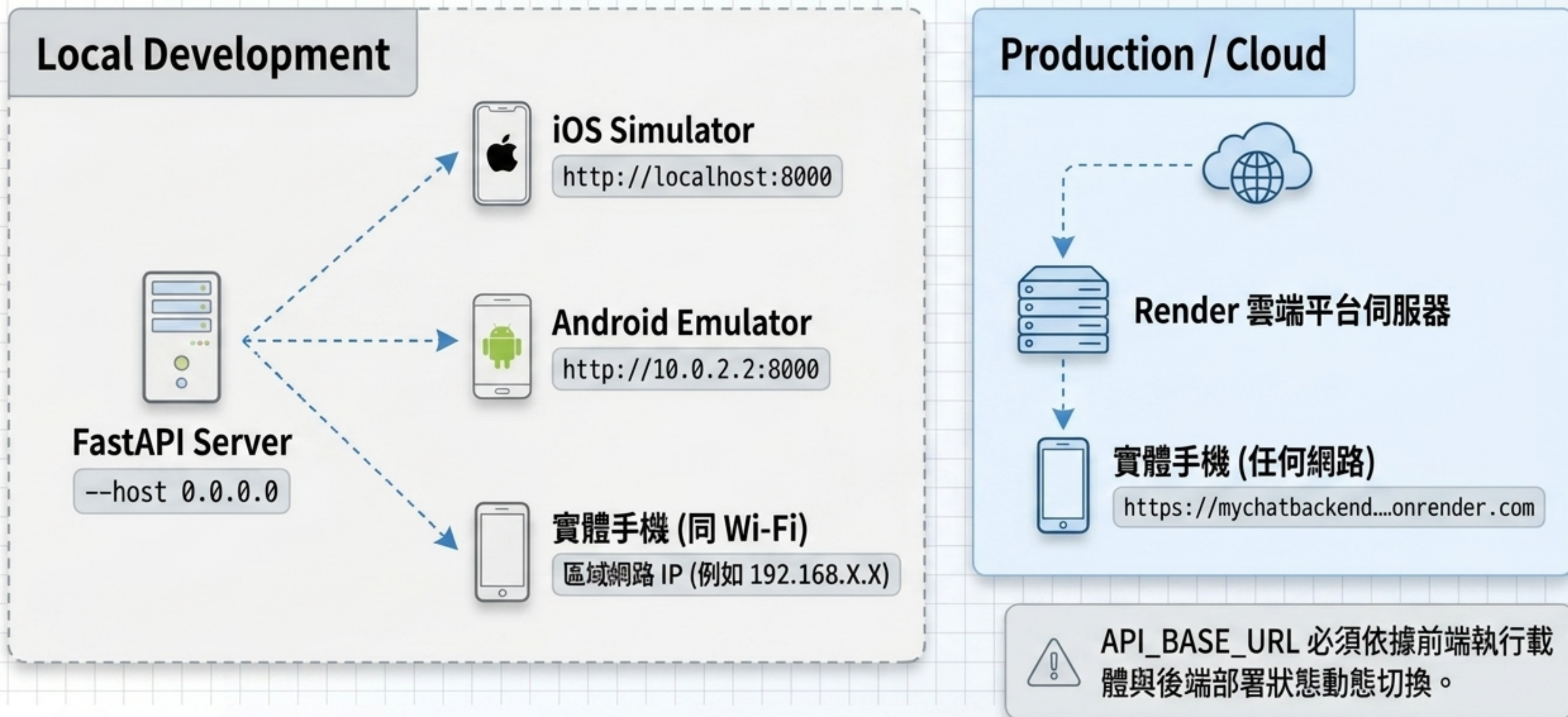
[GET] [POST] /users/{id}/friends 社交關聯

[GET] /users/{id}/chats 對話聚合

[GET] [POST] /chats/{uid}/{fid}/messages 核心互動

REST API 精神不變，但資料模型已從抽象的 Post 轉換為具備實務意義的 User、Friendship 與 Message。

執行環境與網路拓撲：API URL 動態對應



排除連線障礙：Network Request Failed 決策樹

Error: Network request failed

在 apiRequest() 注入探針

→ `console.log('Current API_BASE_URL:', BASE_URL);`

Micro-Terminal

從 Metro Terminal 或 Expo Logs 查看實際印出的網址

> Error: Network request failed
> console.log('Current API_BASE_URL:', BASE_URL);

印出 Localhost?

檢查載體。若是 Android 模擬器必定失敗，需改為 10.0.2.2。

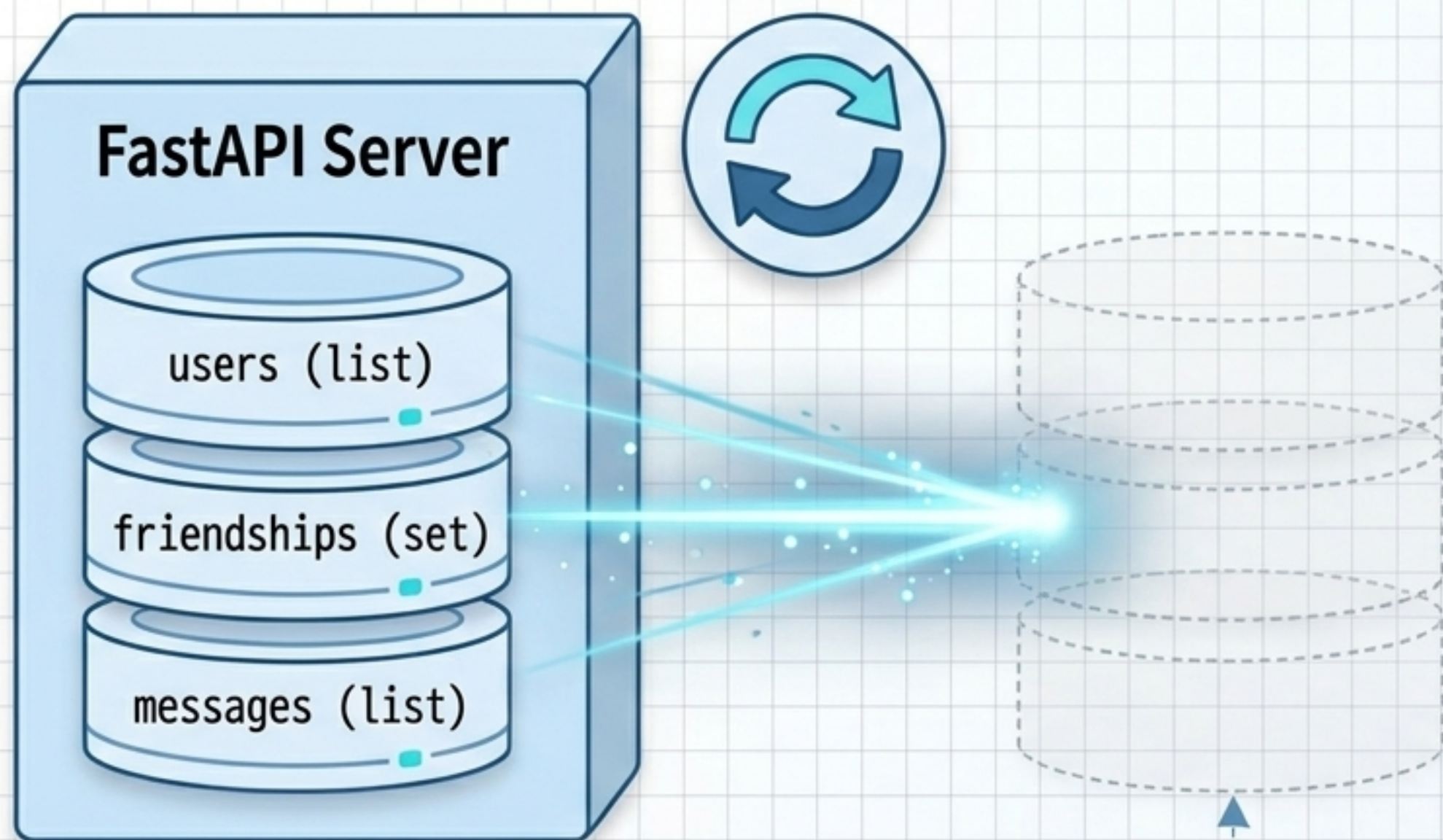
印出區網 IP?

檢查防火牆，並確認手機與電腦是否連上同一 Wi-Fi。

印出部署網址?

確認網址拼寫無誤，且 Render 後端伺服器未處於休眠狀態。

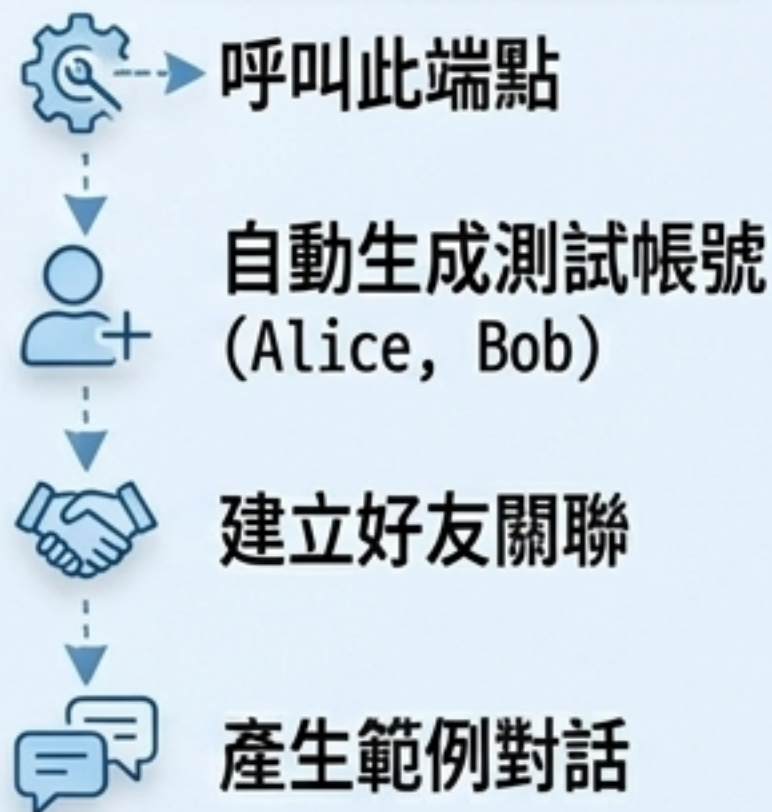
快速原型的代價：揮發性資料庫與 Seed 機制



優點：免安裝、專注 API 資料流動。
限制：重啟即歸零，無法持久化。

開發解法：/dev/seed 端點

⚠ 僅限開發環境使用



解決痛點：避免每次重啟伺服器後，都要反覆進行繁瑣的手動註冊與加好友流程。

身分驗證的現實：從明文比對到 Token 授權

當前範例版 (Prototype / 教學用)

傳遞方式：前端明文傳送帳密。

↓

後端處理：比對記憶體中的明文密碼
→ 直接回傳 `public_user` 物件。

↓

狀態保存：前端純粹依賴 `AuthContext`
存放使用者資訊。

未來正式版 (Production / 工業標準)

傳遞與儲存：密碼進行「單向雜湊
(Hash, 如 `bcrypt`)」後儲存。

↓

後端處理：驗證雜湊成功後，簽發
「JWT Token」。

↓

狀態保存：後端不再信任 URL 參數，
一切權限皆由解析 JWT Token 來判定。

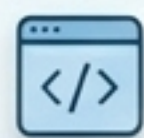
信任邊界 (Trust Boundary)：前端 UX 與後端權限

⚡ Trust Boundary (信任邊界) ⚡

Frontend (React Native AuthContext)



職責：畫面狀態控制 (UX)。



管理 user, signIn, signOut, setUser。



利用 Redirect 隱藏 Tabs，
避免未登入者看到聊天介面。

防君子不防小人，僅提供良好的使用者體驗。

Backend (FastAPI)



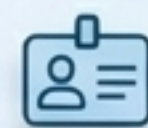
職責：資料權限守門員 (Security)。



不依賴前端狀態，自主檢查：



1. 兩人是否真為好友
(ensure_friends)？

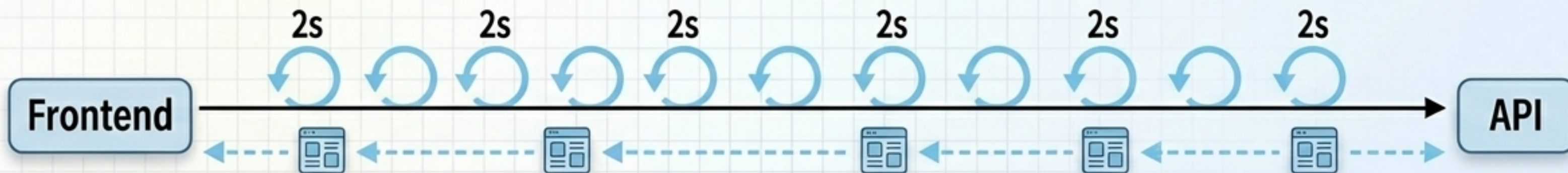


2. 傳送者 ID (sender_id)
是否相符？

真正執行授權規則的地方，決定資料的生殺大權。

突破靜態資料：建立即時互動的對話體驗

⚙️ 當前架構 - 輪詢 (Polling)



週期性主動詢問。適合目前架構，實作成本低。

🔒 未來進階 - 雙向通訊 (WebSocket)



持久連線，伺服器主動推播，沒有時間差。

精準控制背景資源：輪詢的啟動與清除 (Cleanup)

React Native Lifecycle Tracker Diagram

● 進入畫面 (Focus)

- 觸發 `useFocusEffect`
- 執行 `setInterval` → 啟動每2秒撈取資料

[網路請求持續進行中...]



● 離開畫面 (Unfocus / Blur)

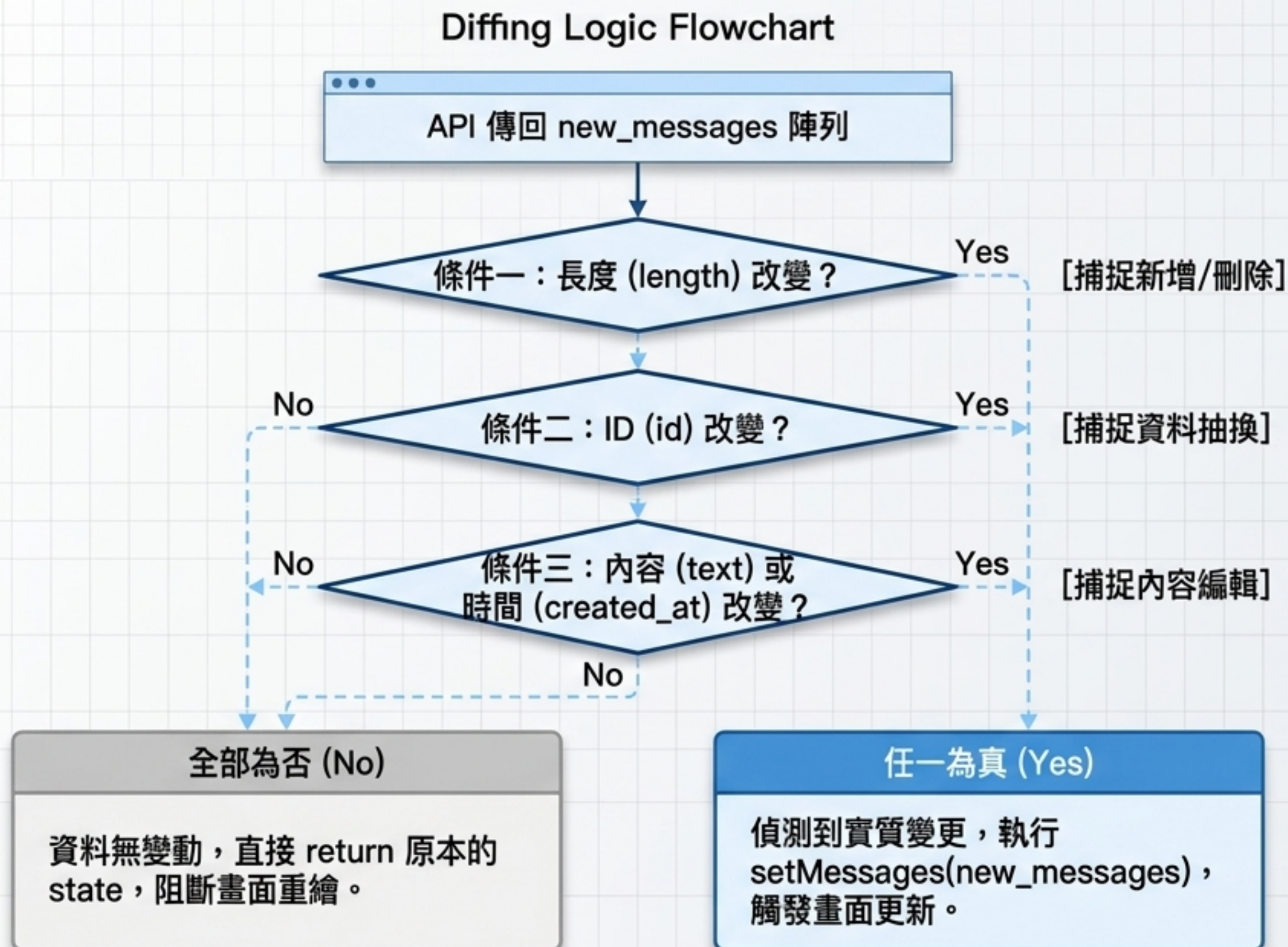
- 觸發 `Return Cleanup Function`
- 執行 `clearInterval` → 強制停止輪詢

[背景任務已終止，釋放記憶體與網路資源]



Key Note：若忘記 Cleanup，會導致背景堆疊大量重複請求，拖垮 App 效能甚至造成崩潰。

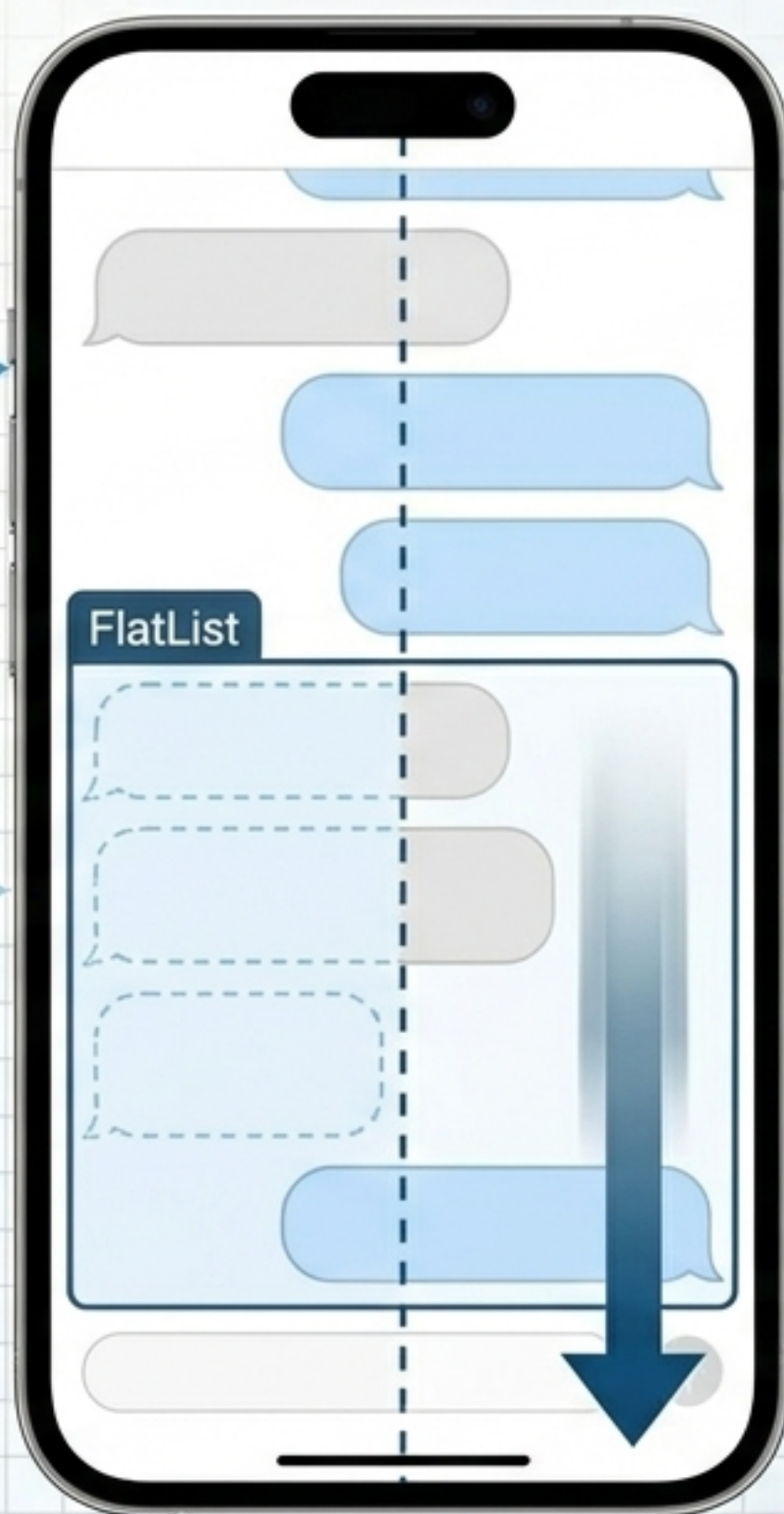
渲染優化：精準攔截重複資料的邏輯閘



UX 細節打磨：自動捲動至最新訊息

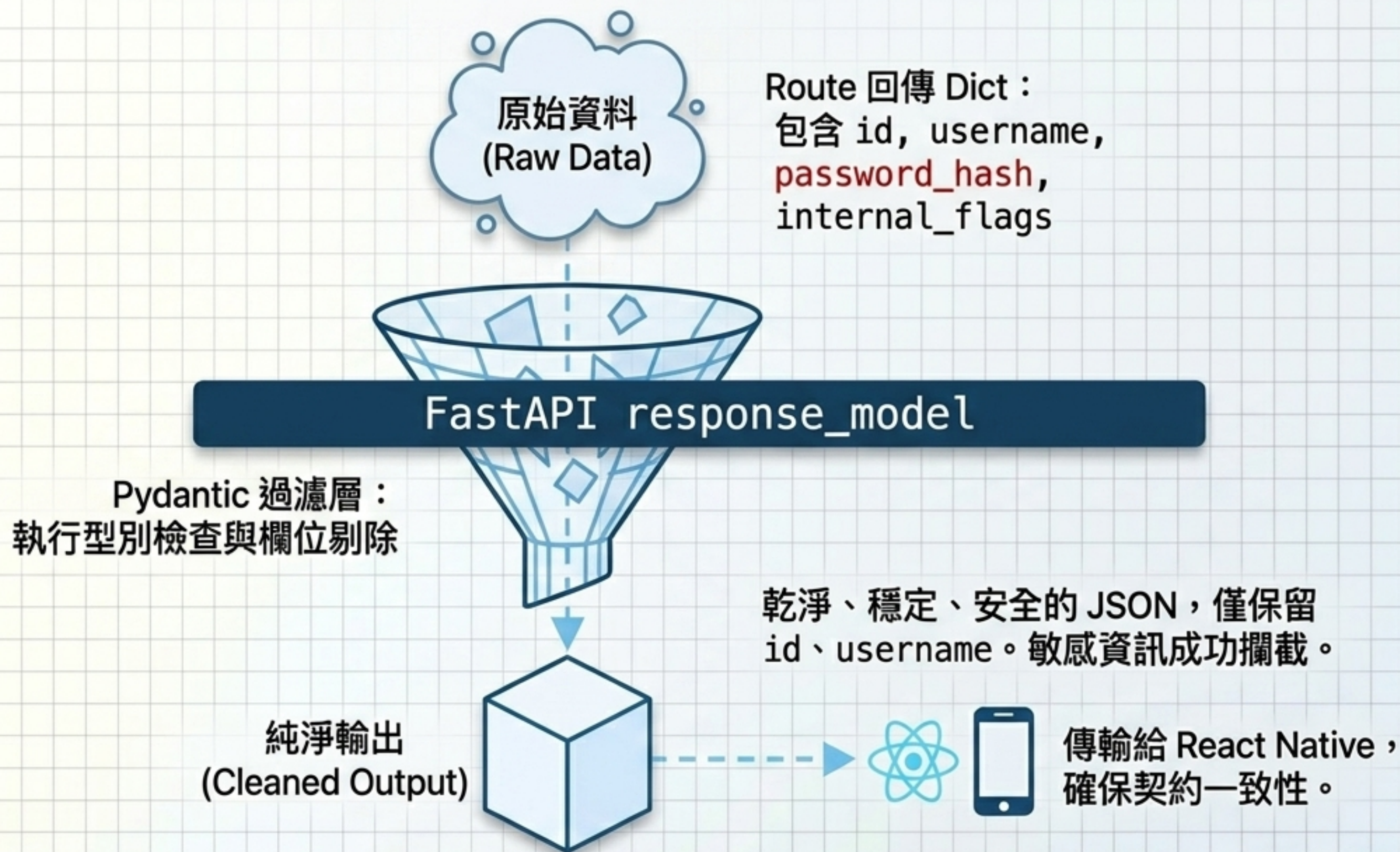
Step 1: messages 狀態改變
→ 觸發 useEffect。

Step 2:
呼叫 requestAnimationFrame
→ 確保 React 已完成 DOM/原生
視圖的渲染。



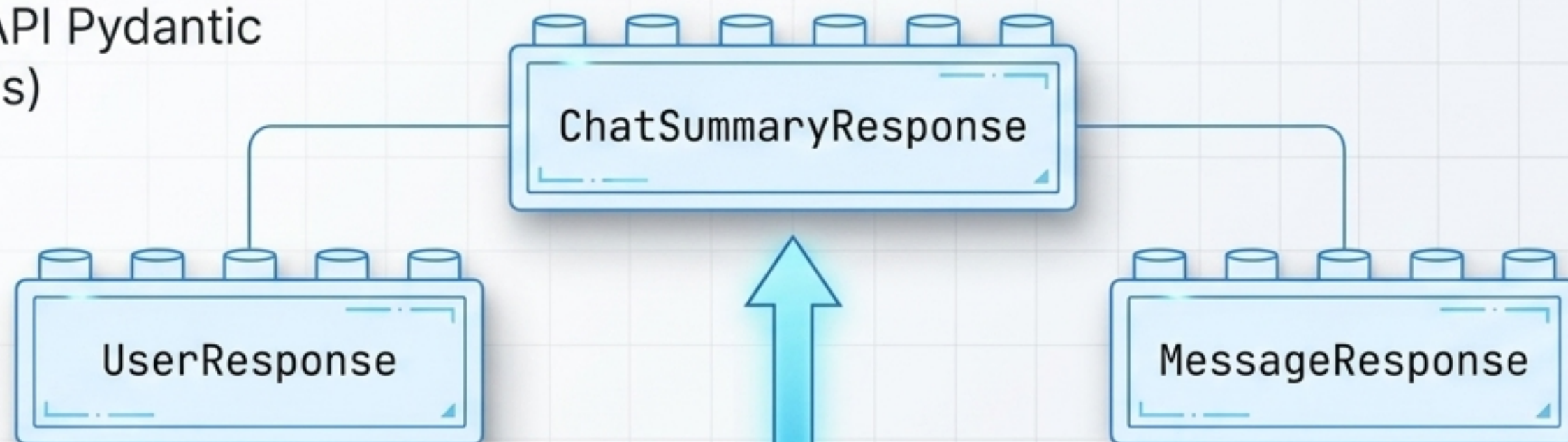
Step 3:
透過 useRef 綁定列表，
執行 scrollToEnd({
 animated: true })。

穩定 API 契約：Response Model 的防護網

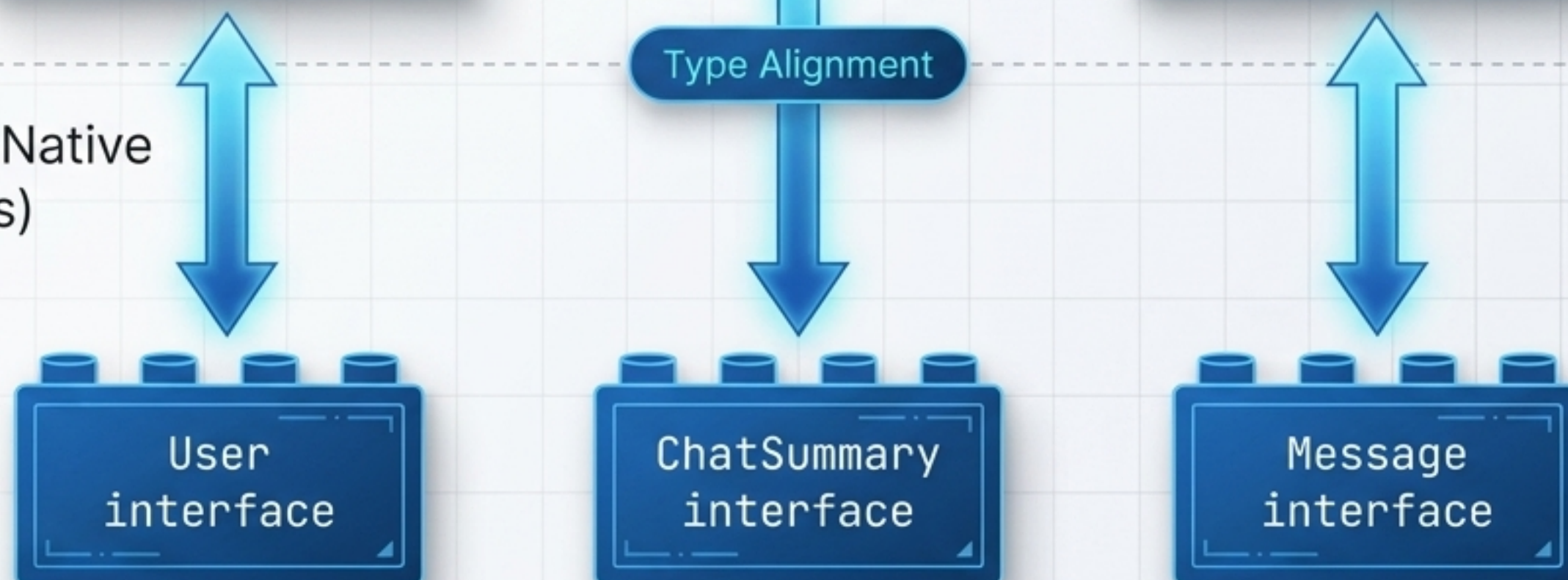


型別系統的對齊：構建模組化資料結構

Backend (FastAPI Pydantic
Pydantic Models)



Frontend (React Native
TypeScript Types)



邁向正式產品：下一階段技術升級藍圖

